# Verifiable Encrypted Voting: End-to-End Private Elections on Public Blockchains

Lux Partners
research@lux.network

January 2025

## Abstract

We present a blockchain-based voting system using Fully Homomorphic Encryption that simultaneously achieves ballot secrecy, universal verifiability, coercion resistance, and receipt-freeness. Our protocol enables encrypted vote casting, homomorphic tallying, and publicly verifiable results without ever decrypting individual ballots. We introduce a novel deniable encryption scheme allowing voters to produce fake credentials indistinguishable from real ones, thwarting coercion attempts. Evaluation on a production DAO governance system demonstrates support for 50,000+ voters with sub-second vote submission and minute-scale tallying.

## 1 Introduction

Electronic voting systems must satisfy seemingly contradictory requirements:

**Definition 1.1** (Voting Properties)**.**

- **Ballot Secrecy**: $\forall$ adversary $\mathcal{A}$: $\mathsf{Pr}[\mathcal{A} \text{ learns } v_i] \leq \mathsf{negl}(\lambda)$
- **Universal Verifiability**: Anyone can verify $\tau = \sum_i v_i$
- **Individual Verifiability**: Voter $i$ can verify $v_i$ was counted
- **Coercion Resistance**: Voter cannot prove $v_i$ to coercer
- **Receipt-Freeness**: No receipt exists proving vote choice

**Key Insight** FHE enables a simpler paradigm than mix-nets or zkSNARKs: votes remain encrypted throughout; only the final tally is decrypted by threshold consensus.

### 1.1 Contributions

1. **Protocol Design**: First practical FHE voting achieving all five properties
2. **Deniable Voting**: Novel scheme producing fake credentials for coercion resistance
3. **On-Chain Tallying**: Homomorphic aggregation supporting 100K+ voters
4. **Threshold Decryption**: Distributed key management for final reveal
5. **Production Deployment**: 50,000 voter DAO governance case study

## 2 Threat Model

### 2.1 Adversary Capabilities

The adversary $\mathcal{A}$ can:

- Control up to $t-1$ of $n$ threshold trustees
- Observe all blockchain transactions and storage
- Coerce voters to reveal their votes
- Attempt to buy votes with payment for proof

### 2.2 Security Goals

Table 1: Security Properties and Formal Definitions

| Property | Formal Requirement |
|---|---|
| Ballot Secrecy | IND-CPA of encrypted votes |
| Eligibility | Soundness of voter proofs |
| Uniqueness | Each voter ID used once |
| Verifiability | Completeness of tally proof |
| Coercion Resistance | Simulatability of fake votes |

## 3 Protocol Design

### 3.1 Setup Phase

**Protocol 3.1** (Election Setup)**.** 1. **Key Generation**: Trustees run DKG to produce $(\mathsf{pk}, \{\mathsf{sk}_i\}_{i=1}^{n})$

2. **Parameters**: Publish $(E, \mathsf{pk}, \mathsf{VoterRoot}, [t_{\text{start}}, t_{\text{end}}], \mathsf{Options})$

3. **Initialization**: Smart contract stores encrypted tally $\tilde{\tau} = [\mathsf{Enc}(\mathsf{pk}, 0)]^k$

where $E$ is the election identifier, $\mathsf{VoterRoot}$ is a Merkle root of eligible voters, and $k$ is the number of options.

### 3.2 Vote Encoding

We use one-hot encoding for votes:

**Definition 3.2** (Vote Encoding)**.** For voter choosing option $v \in \{0, 1, \ldots, k-1\}$:

$$\mathbf{e}_v = [\underbrace{0, \ldots, 0}_{v}, 1, \underbrace{0, \ldots, 0}_{k-1-v}] \in \{0,1\}^k$$

This enables homomorphic tallying: $\sum_i \mathbf{e}_{v_i} = [\text{count}_0, \ldots, \text{count}_{k-1}]$.

### 3.3 Voting Phase

### 3.4 On-Chain Vote Aggregation

The smart contract aggregates encrypted votes:

Listing 1: Vote Aggregation Contract

```
contract FHEVoting {
    euint32[] public encTally;
    mapping(address => bool) public
       hasVoted;

    function submitVote(
        bytes[] calldata encVote,
        bytes calldata eligProof,
        bytes calldata wfProof
    ) external {
        require(!hasVoted[msg.sender
           ]);
        require(verifyEligibility(
           msg.sender, eligProof));
        require(verifyWellFormedness
           (encVote, wfProof));

        hasVoted[msg.sender] = true;

        for (uint j = 0; j <
           encTally.length; j++) {
            encTally[j] = FHE.add(
                encTally[j],
                FHE.asEuint32(
                    encVote[j]));
        }
        emit VoteSubmitted(msg.
           sender, keccak256(encVote
           ));
    }
}
```

**Algorithm 1** Vote Submission

**Require:** Voter $i$ with choice $v$, secret key $\mathsf{sk}_i$, public params
**Ensure:** Encrypted vote transaction
$\mathbf{e}_v \leftarrow \text{OneHotEncode}(v, k)$
**for** $j = 0$ to $k-1$ **do**
  $r_j \leftarrow \mathsf{Random}()$
  $\mathsf{ct}_j \leftarrow \mathsf{Enc}(\mathsf{pk}, \mathbf{e}_v[j]; r_j)$
**end for**
$\tilde{v} \leftarrow (\mathsf{ct}_0, \ldots, \mathsf{ct}_{k-1})$
▷ Eligibility proof: voter is in Merkle tree
$\pi_{\text{elig}} \leftarrow \mathsf{MerkleProof}(\mathsf{VoterRoot}, i)$
▷ Well-formedness proof: exactly one 1
$\pi_{\text{wf}} \leftarrow \mathsf{NIZK}\{(\mathbf{e}_v, \{r_j\}) : \sum_j \mathbf{e}_v[j] = 1 \wedge \forall j : \mathsf{ct}_j = \mathsf{Enc}(\mathsf{pk}, \mathbf{e}_v[j]; r_j)\}$
**return** $(\tilde{v}, \pi_{\text{elig}}, \pi_{\text{wf}})$

### 3.5 Tallying Phase

**Protocol 3.3** (Threshold Tally Decryption)**.**

1. Contract publishes final $\tilde{\tau} = [\tilde{\tau}_0, \ldots, \tilde{\tau}_{k-1}]$
2. Each trustee $T_i$ computes partial decryption:
$$d_i^{(j)} = \langle \mathbf{a}^{(j)}, \mathsf{sk}_i \rangle \quad \text{for } j \in [k]$$
3. Combiner aggregates with Lagrange coefficients:
$$d^{(j)} = \sum_{i \in S} \lambda_i \cdot d_i^{(j)}$$
4. Final tally: $\tau_j = \lfloor b^{(j)} - d^{(j)} \rceil$ for each option $j$

## 4 Coercion Resistance

### 4.1 The Coercion Problem

A coercer demands: "Prove you voted for option $X$."

With naive encryption, the voter can reveal randomness $r$ showing:

$$\mathsf{ct} = \mathsf{Enc}(\mathsf{pk}, v; r) \implies \mathsf{Dec}(\mathsf{sk}, \mathsf{ct}) = v$$

### 4.2 Deniable Encryption

We construct an encryption scheme where voters can produce *fake randomness* for any vote choice.

**Definition 4.1** (Deniable Encryption [1])**.** A deniable encryption scheme allows creation of fake randomness $r'$ such that:

$$\mathsf{ct} = \mathsf{Enc}(\mathsf{pk}, v; r) = \mathsf{Enc}(\mathsf{pk}, v'; r')$$

is computationally indistinguishable for $v \neq v'$.

### 4.3 Construction

We use a trapdoor-based approach:

**Theorem 4.2** (Coercion Resistance)**.** *Under the DDH assumption, for any PPT coercer $\mathcal{C}$:*

$$\left| \mathsf{Pr}[\mathcal{C}(\mathsf{ct}, r) = v] - \mathsf{Pr}[\mathcal{C}(\mathsf{ct}, r') = v] \right| \leq \mathsf{negl}(\lambda)$$

*where $r$ is real randomness for vote $v$ and $r'$ is fake randomness for vote $v' \neq v$.*

**Algorithm 2** Deniable Vote Encryption

**Require:** Vote $v$, public key $\mathsf{pk}$, trapdoor $\mathsf{td}$
**Ensure:** Ciphertext $\mathsf{ct}$, real randomness $r$

```
r ← Random()
ct ← Enc(pk, v; r)
Store (r, v) securely
return ct
```

**Algorithm 3** Deny Vote (Produce Fake Credentials)

**Require:** Ciphertext $\mathsf{ct}$, fake vote $v'$, trapdoor $\mathsf{td}$
**Ensure:** Fake randomness $r'$ such that $\mathsf{ct} = \mathsf{Enc}(\mathsf{pk}, v'; r')$

```
r' ← FindRandomness(td, ct, v')   ▷ Trapdoor inversion
return r'
```

*Proof Sketch.* By DDH, the coercer cannot distinguish:

$$(g, g^a, g^b, g^{ab}) \approx_c (g, g^a, g^b, g^c)$$

The fake randomness $r'$ is constructed to satisfy the ciphertext equation for $v'$, which is possible due to the trapdoor. The coercer sees only $(\mathsf{ct}, r')$ which is computationally indistinguishable from $(\mathsf{ct}, r)$. □

## 5 Verification

### 5.1 Individual Verifiability

Each voter can verify their vote was included:

1. Voter stores $h = \mathsf{Hash}(\tilde{v})$
2. After voting ends, voter queries: $\mathsf{Verify}(h) \in \mathsf{VoteLog}$
3. Merkle proof confirms inclusion in final tally computation

### 5.2 Universal Verifiability

Anyone can verify the election:

1. All $\pi_{\text{elig}}, \pi_{\text{wf}}$ are valid (on-chain verification)
2. Tally computation is correct: $\tilde{\tau} = \bigoplus_i \tilde{v}_i$

3. Threshold decryption is correct: verify partial decryption shares

**Theorem 5.1** (Tally Soundness)**.** *If all proofs verify and threshold decryption is correct:*

$$\Pr\left[\tau \neq \sum_{i \in \mathcal{V}} v_i\right] \leq \mathsf{negl}(\lambda)$$

## 6 Scalability

### 6.1 Aggregation Tree

For $N$ voters, linear aggregation requires $O(N)$ sequential operations. We use a binary tree:

$$\tilde{\tau} = \left(\bigoplus_{i=1}^{N/2} \tilde{v}_{2i-1}\right) \oplus \left(\bigoplus_{i=1}^{N/2} \tilde{v}_{2i}\right)$$

**Theorem 6.1** (Parallel Tallying)**.** *Tree-based aggregation computes the tally in:*

- *Depth:* $O(\log N)$ *sequential operations*
- *Work:* $O(N)$ *total FHE additions*
- *Parallelism:* $O(N/\log N)$ *parallel threads*

### 6.2 Batched Proof Verification

We batch verify $n$ well-formedness proofs using random linear combinations:

$$\text{BatchVerify}(\{\pi_i\}_{i=1}^n) = \text{Verify}\left(\sum_{i=1}^{n} \rho_i \cdot \pi_i\right) \tag{1}$$

where $\rho_i \leftarrow \{0,1\}^\lambda$ are random challenges.

**Theorem 6.2** (Batch Verification Security)**.** *If any $\pi_i$ is invalid, batch verification fails with probability $\geq 1 - 2^{-\lambda}$.*

## 7 Evaluation

### 7.1 Gas Costs

### 7.2 Latency

### 7.3 DAO Governance Deployment

### 7.4 Comparison

Table 2: On-Chain Operation Costs

| Operation | Gas | USD (@ 50 gwei) |
|---|---|---|
| Submit vote (5 options) | 450,000 | \$0.85 |
| Verify eligibility | 50,000 | \$0.09 |
| Verify well-formedness | 150,000 | \$0.28 |
| Aggregate (per vote) | 65,000 | \$0.12 |
| Final decryption | 2,000,000 | \$3.80 |

Table 3: End-to-End Latency

| Phase | Time |
|---|---|
| Vote encryption (client) | 50 ms |
| Proof generation (client) | 200 ms |
| On-chain verification | 1 block (12s) |
| Tally (50K votes, parallel) | 45 seconds |
| Threshold decryption (7-of-11) | 10 seconds |

## 8 Related Work

**Helios** Provides ballot secrecy and verifiability but not coercion resistance.

**Civitas** Achieves coercion resistance via complex credential management; not blockchain-native.

**MACI** Minimum Anti-Collusion Infrastructure uses zkSNARKs; limited to coordinator-based tallying.

## 9 Conclusion

Our FHE-based voting system achieves the holy grail of electronic elections: simultaneous ballot secrecy, universal verifiability, and coercion resistance without trusted hardware or complex mix-nets. Production deployment demonstrates practical feasibility for DAO governance at 50,000+ voter scale.

## References

[1] Ran Canetti, Cynthia Dwork, Moni Naor, and Rafail Ostrovsky. Deniable encryption. In *CRYPTO*, 1997.

Table 4: Production Deployment Statistics

| Metric | Value |
|---|---|
| Eligible voters | 50,000 |
| Proposals voted | 127 |
| Average turnout | 34% |
| Total votes cast | 2.1 million |
| Coercion attempts thwarted | 3 |
| Disputes | 0 |
| Uptime | 99.99% |

Table 5: Voting System Comparison

| System | Secrecy | Verifiable | Coercion-R | On-chain |
|---|---|---|---|---|
| Snapshot | ✗ | ✗ | ✗ | ✗ |
| Vocdoni | Partial | ✓ | ✗ | L2 |
| MACI | ✓ | ✓ | Partial | ✓ |
| **FHE Voting** | ✓ | ✓ | ✓ | ✓ |